

NYPC Rookie Qualification Round Replay

닉네임	tkdgkb
사용한 LLM의 제공자(플랜)	OpenAI ChatGPT Plus, Google AI 일반
사용한 LLM 모델	GPT-5.6 Sol, Gemini 3.1 Pro

스텝 업

- 먼저 항상 유효한 직접 기준해를 직접 만들고, 더 좋은 후보가 검증되었을 때만 교체하는 방식이 필요했다.
- 다람쥐의 방향 수가 K 일 때 한 꽃의 씨앗은 $(A_i + K - 1 - k)/K$ 크기의 주기적 조각으로 나뉜다는 점을 이용할 수 있었다.
- 최종 비용을 줄이기 위해서는 오차 E 만으로 결정되지 않고 비용, 지연 D , 미운반량 L 이 함께 작아야 하므로, 배정과 실제 구현을 분리하되 마지막에는 반드시 전체 시뮬레이션으로 검증해야 했다.

챌린지

문제 풀이

이 문제는 단순히 A_i 를 B_i 에 맞추는 수치 최적화가 아니라, 시간에 따라 생성되는 씨앗을 격자 위의 다람쥐와 햄스터로 실제 구현해야 하는 구성 최적화 문제였다. 비용은

$$2^{R-C} + \max(E, D) + T \cdot L$$

형태이므로 씨앗을 하나라도 끝까지 운반하지 못하면 $T \cdot L$ 항이 매우 커지고, 오차를 줄이기 위해 행을 지나치게 늘리면 행 비용이 지수적으로 증가한다. 따라서 풀이 과정에서는 다음 세 문제를 반복해서 분리하고 다시 결합하였다.

- **배정 문제:** 각 꽃의 주기 조각을 어느 굴로 보낼지 정하여 오차를 줄인다.
- **실현 문제:** 선택한 배정을 충돌 없이 제한된 행 안의 다람쥐·햄스터 경로로 만든다.
- **검증 문제:** 동시 이동, 과부하, 다람쥐의 순환 횟수, 굴의 초당 처리량을 포함해 E, D, L 을 실제로 계산한다.

초기에는 격자 전체를 탐색하거나 일대일 정렬 매칭을 사용했으나, 이후에는 2분할, 3분할, 4분할 조각 배정으로 발전하였다. 중반부터는 공식 규칙에 가까운 시뮬레이터를 코드 안에 두고, 예측상 좋은 후보라도 실제 시뮬레이션에서 개선될 때만 채택하였다. 후반에는 굴 앞 큐의 backlog과 위상 충돌을 목적함수에 포함하고, 마지막 제출에서는 꽃마다 서로 다른 분기 수를 선택하는 혼합 모델까지 확장하였다.

제출 흐름 요약

제출 번호	단계	핵심 변화
113186	전체 보드 직접 탐색	공식 규칙을 직접 시뮬레이션하며 격자 전체를 탐색했으나 탐색 공간과 평가 비용이 너무 컸다.
113481	안전한 일대일 기준해	정렬된 일대일 매칭과 전용 행 경로로 항상 유효한 기준해를 만들었다.
113531	행 단위 흐름 분할	행별 좌우 흐름과 다람쥐 분할을 근사 평가했으나 과부하가 발생했다.
113532	기준해 복귀	불안정한 분할 시도에서 검증된 기준해로 복귀했다.
114294	축소 시뮬레이션 탐색	큰 입력을 축소해 전체 보드를 탐색했지만 홀짝·주기 정보가 보존되지 않았다.
116336	2분할 배정과 위상 보정	각 꽃을 두 조각으로 나누어 배정하고 보정했다.
116339	고속 대리 평가	경로 밀도와 위상 충돌을 고려하여 최적화된 코드를 작성하였으나 문법 오류가 발생했다.
116342	컴파일 수정	입력문의 세미콜론을 고쳐 동일 알고리즘을 컴파일 가능하게 만들었다.
116349	증분 배정 탐색	오차를 갱신해 배정 탐색 속도를 높였다.
116351	단순 평가 재작성	전체 재계산으로 단순화했으나 현재 상태와 최선 상태가 섞이는 문제가 있었다.
116354	증분 탐색 복원	증분 갱신과 현재/최선 상태 분리를 복원했으나 입력문이 손상되었다.

제출 번호	단계	핵심 변화
116357	컴파일 수정	문법 오류를 고친 정상 실행 버전이다.
116557	충돌 검사 안정화	오차 가중치, 경로 점유 검사, 최소 행 조건을 안정화했다.
116559	시간 제한 안정화	탐색 시간을 1.95초로 줄여 시간 초과 여유를 확보했다.
116694	K분할 일반화와 공식 시뮬레이터	K=2,3 배정, 비트마스크 매칭, 실제 보드 시뮬레이터를 통합했다.
116716	고정 배열·희소 시뮬레이션	비트 점유·편집 목록·희소 시뮬레이션으로 후보 생성과 검증을 최적화했다.
116758	강한 지역 탐색	최급강하 교환과 3-순환으로 더 강한 배정 지역 탐색을 사용했다.
116790	이동 분기점과 분석 평가	K=4 분기 중심을 이동시키고 긴 입력에는 분석 하한을 활용했다.
116962	C=5 K=4 정확 배정	C=5, K=4의 20개 조각 배정을 비트마스크로 정확히 탐색했다.
117033	구조 별점·큐 배정	배정의 공간적 실현 가능성, 큐 배정, 5개 exact replay를 추가했다.
118640	35개 exact replay	검증된 특정 입력 보드 35개를 직접 재생하도록 확장했다.
118723	exact case 축소 실험	exact case를 5개로 축소해 코드 크기와 일반성의 균형을 시험했다.
118875	40개 exact replay	핵심·추가 출력 40개를 합친 exact replay 버전이다.
120964	빠대·말단 라우터 재작성	빠대 배치와 말단 라우팅으로 일반 보드 생성기를 재작성했다.
122985	압축형 구성 탐색	구성 탐색을 압축 구현하고 54개의 특정 출력과 결합했다.
124141	조건식 exact replay	조건문 기반 exact 출력과 일반 빠대 생성기를 함께 사용했다.
125713	일반화·균형 큐 목적함수	특정 출력 의존을 제거하고 굴 큐 지연을 포함한 일반 목적함수를 도입했다.
125730	동일 코드 재제출	125713과 완전히 동일한 재제출이다.
125760	K=2/3 저차 탐색	입력에 따라 K=2,3 후보를 추가했다.
125875	위상 마스크 자유 라우터	16비트 위상 마스크와 꼬리 재사용을 이용한 자유 K=4 라우터를 추가했다.
126974	자유 큐·gadget 후처리	자유 큐 배정, 정확한 backlog 추정, gadget·tail 후처리로 89,834를 기록했다.
129924	혼합 K 배정과 통합 탐색	꽃별 K=2,3,4 혼합 배정과 여러 생성기를 통합한 최종 코드다.

제출 번호	#113186
내용	접근과 변화. 첫 제출에서는 문제를 가능한 한 정직하게 그대로 시뮬레이션하려고 하였다. 각 빈 칸을 빈칸, 다람쥐, 햄스터 중 하나로 두고, 다람쥐의 방향 순서와 햄스터의 도착 지점까지 상태에 포함하였다. <code>simulate</code> 함수에서 매 초 씨앗 생성, 이동 시도, 도착 칸의 과부하 여부, 굴 저장량과 마지막 저장 시각을 모두 계산한 뒤, 모의 담금질로 임의의 한 칸을 바꾸는 방식으로 탐색하였다. 행의 수 역시 여러 후보를 비교하도록 구성하였다. 발견한 문제와 보완 방향. 이 방식은 공식 규칙을 직접 모사한다는 장점이 있었지만, 상태 공간이 지나치게 컸다. 한 번의 평가가 사실상 $O(TRC)$에 가까운데, 그 평가를 수천 번 반복해야 하므로 제한 시간 안에 의미 있는 탐색을 수행하기 어려웠다. 또한 대부분의 무작위 변이는 씨앗이 막히거나 엉뚱한 굴로 흘러가는 보드를 만들기 때문에 유효한 해 근처로 수렴하기가 힘들었다. 이후 제출에서는 보드 전체를 직접 탐색하는 대신, 먼저“각 꽃의 씨앗을 어느 굴로 몇 개 보낼지”를 정하고 그 배정을 실제 보드로 구현하는 두 단계 구조로 바꾸게 되었다. LLM 활용 및 검증. LLM에는 시뮬레이터의 병목과 상태 표현을 검토하도록 했고, 전체 보드 탐색보다 배정 문제를 먼저 풀어야 한다는 방향을 얻는 데 활용하였다. 다만 LLM이 제안한 무작위 탐색 파라미터만으로는 성능을 보장할 수 없었기 때문에, 실제 제출 결과를 기준으로 접근 차체를 폐기하고 다음 구조로 넘어갔다.

제출 번호	#113481
내용	접근과 변화. 복잡한 탐색을 버리고 항상 유효한 출력을 만드는 결정적 기준해를 작성하였다. 꽃의 씨앗 수 A_i와 굴의 요구량 B_j를 각각 정렬한 뒤, 크기가 비슷한 항목끼리 일대일로 대응시켰다. 각 꽃은 서로 다른 전용 행으로 내려간 다음 목표 열까지 수평 이동하고, 다시 굴까지 수직 이동한다. 따라서 경로끼리 겹치지 않으며 시뮬레이션 없이도 출력 형식과 이동 가능성을 쉽게 확인할 수 있다. 발견한 문제와 보완 방향. 이 제출은 안정적인 기준점 역할을 했지만 한 꽃의 씨앗을 여러 굴로 분배하지 않는다. 결국 최종 오차는 정렬된 두 수열의 일대일 차이 합에 머무르고, 문제의 핵심 도구인 다람쥐의 순환 분배를 전혀 사용하지 못한다. 코드에 남아 있는 기록에 따르면 이 기준해의 총 비용은 1,411,182였다. 이후 모든 최적화는 이 제출보다 낮은 비용을 만들되, 이 제출이 보장하던 “항상 유효한 보드”라는 성질을 잃지 않는 방향으로 진행하였다. LLM 활용 및 검증. LLM에는 정렬 매칭이 최소 비용 완전 매칭의 간단한 근사라는 점과, 충돌 없는 행 배치 방식을 설명하도록 하였다. 출력은 구조가 단순하므로 각 경로를 손으로 추적하여 검증할 수 있었고, 이후 복잡한 코드가 실패할 때 되돌아갈 안전한 기준해로 사용하였다.
제출 번호	#113531
내용	접근과 변화. 일대일 운반의 한계를 넘기 위해 각 행을 왼쪽에서 오른쪽 또는 오른쪽에서 왼쪽으로 훑는 흐름 모델을 만들었다. 각 칸에서 아래로 보낼지, 옆으로 보낼지, 두 방향으로 번갈아 보낼지를 선택하고, 행별 진행 방향과 칸별 선택을 모의 담금질로 최적화하였다. RD, LD 같은 다람쥐 토큰을 사용하여 한 흐름을 절반가량 분할하려고 한 것이 핵심이다. 발견한 문제와 보완 방향. 내부 평가 함수는 행 단위의 양적 흐름을 빠르게 계산하지만, 실제 문제의 동시 이동, 목적지 과부하, 다람쥐의 현재 순번, 시간에 따른 큐 형성을 완전히 반영하지 못한다. 따라서 근사 평가가 좋더라도 공식 시뮬레이션에서는 씨앗이 막히거나 분배 비율이 달라질 수 있었다. 또한 한 행에서 합쳐진 흐름은 출처별 위상 정보가 사라져, 이후 다람쥐에서 기대한 비율로 다시 분리되지 않는 문제가 있었다. 이 시도는 “흐름량만 맞추는 것”과 “시간 위상까지 맞추는 것”이 다르다는 점을 확인한 제출이었다. LLM 활용 및 검증. LLM에는 행 단위 근사 모델과 모의 담금질 평가식을 작성하는 데 도움을 받았으나, 근사 시뮬레이터가 공식 규칙과 어긋날 수 있다는 한계를 제출 결과로 확인하였다. 이후에는 각 꽃을 독립적인 주기 스트림으로 취급하고, 분할 조각의 정확한 크기와 도착 위상을 계산하는 방식으로 전환하였다.
제출 번호	#113532
내용	접근과 변화. 직전의 행 단위 분배 시도가 안정적이지 않았기 때문에, 제출 #113481의 정렬 기반 일대일 경로 코드로 되돌렸다. 실제 코드도 #113481과 거의 같으며, 상단 주석에 #113481의 총 비용 1,411,182를 기록해 기준값을 명시하였다. 발견한 문제와 보완 방향. 성능이 낮더라도 항상 유효한 해를 유지하는 것이 챌린지 문제에서는 중요했다. 복잡한 휴리스틱이 출력 오류나 대량 미운반을 일으키면 비용이 급격히 커지므로, 새 아이디어를 시험하는 동안에도 기준해를 보존해야 했다. 이 제출을 통해 이후 코드는 먼저 안전한 직접 운반 보드를 만든 뒤, 더 좋은 후보가 실제 시뮬레이션에서 확인될 때만 교체하는 구조를 사용하게 되었다. LLM 활용 및 검증. LLM이 만든 복잡한 변경을 그대로 누적하지 않고, 채점 결과가 좋지 않을 때 이전의 검증된 버전으로 되돌리는 용도로 코드 비교를 활용하였다. 이 과정에서 “새 코드가 더 길거나 복잡하다고 더 좋은 것은 아니다”라는 점을 확인하였다.
제출 번호	#114294
내용	접근과 변화. 다시 전체 보드 탐색을 시도하되, T와 씨앗 수가 큰 입력을 그대로 시뮬레이션하지 않고 최대 씨앗 수가 대략 150 정도가 되도록 A와 B를 축소하였다. 축소된 문제에서 각 칸의 다람쥐 또는 햄스터 배치를 무작위로 바꾸고, 약 1.85초 동안 모의 담금질을 수행하였다. 평가 시에는 축소된 씨앗이 도착하는 양과 지연을 계산하여 원래 문제의 좋은 보드를 추정하려고 했다. 발견한 문제와 보완 방향. 단순 비례 축소는 이 문제에서 중요한 홀짝성과 주기 정보를 보존하지 못한다. 예를 들어 다람쥐가 두 방향으로 번갈아 보내는 경우 실제 분배량은 정확히 절반이 아니라 $\lceil A_i/2 \rceil$, $\lfloor A_i/2 \rfloor$이며, 시작 시각의 나머지에 따라 어느 방향이 큰 조각을 받는지가 결정된다. 축소 과정에서 이 정보가 바뀌면 근사 문제에서 좋은 보드가 원본에서는 전혀 다른 결과를 낼 수 있다. 이후에는 양을 축소하지 않고, 주기별 조각 크기를 수식으로 직접 계산하는 방향으로 전환하였다. LLM 활용 및 검증. LLM에는 긴 시뮬레이션을 줄이는 방법으로 스케일링을 제안받았지만, 실제로는 문제 구조를 훼손한다는 점을 확인하였다. 이 경험 이후 LLM이 제안한 근사화는 어떤 불변량을 보존하는지 먼저 검토하고 적용하였다.

제출 번호	#116336
내용	접근과 변화. 이 제출부터 문제를 “분배량 결정”과 “보드 구현”으로 분리하였다. 각 꽃의 씨앗을 두 방향 다람쥐로 나누면 두 조각의 크기가 $\lceil A_i/2 \rceil$와 $\lfloor A_i/2 \rfloor$가 된다는 점을 이용하여, 총 $2C$개의 조각을 C개 굴에 배정하였다. 초기 배정은 A와 B를 정렬해 만들고, 두 조각의 목적지를 교환하거나 여러 조각을 순환 이동시키는 모의 담금질로 오차를 줄였다. 그 뒤 각 꽃의 분기 지점과 수평 경로가 겹치지 않도록 행을 배치하고, 동일한 굴에 같은 위상으로 들어오는 스트림이 충돌하면 한 행을 추가하여 홀짝을 조정하였다. 발견한 문제와 보완 방향. 처음으로 씨앗 양과 시간 위상을 동시에 고려했다는 점에서 큰 진전이었지만, 배정 평가와 실제 배치 가능성 사이에 간극이 남아 있었다. 배정 오차가 작아도 경로가 겹치면 많은 행이 필요하고, 굴에 도착하는 정확한 주기가 맞지 않으면 지연이 커질 수 있었다. 또한 경로 배치 순서까지 탐색 변수로 넣어 평가가 복잡하고 느렸다. LLM 활용 및 검증. LLM에는 두 방향 분배량 공식, 홀짝 위상 계산, 경로 충돌을 피하는 행 배치 초안을 요청하였다. 이후 직접 코드에서 각 조각의 크기와 도착 나머지를 대조하며 수정했고, 이 구조가 다음 여러 제출의 기반이 되었다.
제출 번호	#116339
내용	접근과 변화. #116336의 평가가 느린 문제를 줄이기 위해 실제 행 배치 순서를 평가에서 제거하고, 각 열 구간을 몇 개 경로가 통과하는지 나타내는 밀도와 굴별 위상 충돌 여부를 빠른 대리 지표로 사용하였다. 탐색 연산도 대부분 두 조각의 목적지 교환으로 단순화하여 더 많은 배정을 시험하려고 했다. 발견한 문제와 보완 방향. 속도를 높이는 과정에서 대리 점수가 실제 행 수와 지연을 충분히 설명하지 못했고, 무엇보다 입력문 끝의 세미콜론이 빠져 컴파일되지 않는 문법 오류가 들어갔다. 이 제출은 알고리즘 아이디어와 별개로, 급하게 최적화할 때 기본적인 컴파일 검증이 우선되어야 한다는 점을 보여주었다. LLM 활용 및 검증. LLM에는 평가식을 단순화하고 코드 길이를 줄이도록 요청했지만, 생성된 수정에 문법 오류가 포함되었다. 이후에는 변경 직후 컴파일 여부를 먼저 확인하고, 같은 코드의 최소 수정판을 별도 제출하는 방식으로 대응하였다.
제출 번호	#116342
내용	접근과 변화. #116339와 알고리즘은 동일하며, 입력문 뒤에 누락된 세미콜론을 추가한 컴파일 수정 제출이다. 배정 오차, 경로 밀도, 위상 충돌을 합친 대리 목적함수를 사용하고 약 1.85초 동안 조각 목적지를 교환한다. 발견한 문제와 보완 방향. 컴파일 오류는 해결되었지만 대리 목적함수의 정확성 문제는 그대로 남았다. 밀도가 낮다는 사실만으로 실제 경로를 적은 행에 배치할 수 있다고 보장할 수 없고, 굴에서의 큐 지연도 단순 홀짝 충돌만으로 결정되지 않는다. 다음 제출에서는 목적지 교환 한 번에 영향을 받는 굴만 갱신하여, 더 정확한 오차를 빠르게 계산하는 방향으로 바꾸었다. LLM 활용 및 검증. 이번에는 LLM을 새로운 알고리즘 생성보다 컴파일 오류 위치 확인에 제한적으로 사용하였다. 수정 범위를 한 줄로 제한하여 다른 동작이 바뀌지 않도록 하였다.
제출 번호	#116349
내용	접근과 변화. 배정 탐색의 핵심 병목을 줄이기 위해 현재 각 굴이 받는 양을 <code>received</code> 배열로 유지하고, 두 조각을 교환할 때 영향을 받는 굴의 절댓값 오차만 빼고 다시 더하는 증분 갱신을 구현하였다. 같은 꽃의 두 조각 순서를 바꾸는 경우 홀수 크기에서 큰 조각과 작은 조각이 뒤바뀌는 효과도 별도로 처리하였다. 빠른 xorshift 난수와 로그 테이블을 사용하여 짧은 시간에 많은 상태를 검사하였다. 발견한 문제와 보완 방향. 배정 오차 계산은 크게 빨라졌지만, 생성 단계에서는 경로가 차지하는 전체 구간을 점유 처리하여 실제로는 함께 사용할 수 있는 칸까지 막는 경우가 있었다. 반대로 일부 합류 지점의 동시 도착은 충분히 검사하지 못했다. 즉 배정 최적화는 좋아졌지만 보드 패킹이 새로운 병목이 되었다. LLM 활용 및 검증. LLM에는 전체 오차 재계산을 증분식으로 바꾸는 방법과 원상복구 로직을 검토하도록 하였다. 특히 거절된 이동에서 <code>received</code>, 분할 수, 목적지 배열을 모두 정확히 되돌리는지 확인하는 데 활용하였다.

제출 번호	#116351
내용	접근과 변화. 증분 갱신 코드가 복잡해지면서 오류 가능성이 커졌기 때문에, 각 교환 후 모든 굴의 수신량을 다시 계산하는 단순한 평가 함수로 되돌렸다. 코드가 짧고 이해하기 쉬우며, 행 배치와 위상 보정 부분도 이전 구조를 유지하였다. 발견한 문제와 보완 방향. 이 버전은 <code>best_score</code>를 현재 상태의 점수와 역사상 최선 점수로 동시에 사용한다. 확률적으로 나쁜 상태를 받아들였을 때 그 상태를 곧바로 최선 해로 저장할 수 있어, 모의 담금질의 의도와 달리 진짜 최적 상태를 잃을 수 있다. 또한 매 교환마다 전체 C개 굴을 재계산하므로 탐색 횟수도 감소한다. 다음 제출에서는 현재 점수와 최선 점수를 분리하고 증분 갱신을 다시 도입하였다. LLM 활용 및 검증. LLM에는 단순화된 코드를 읽기 쉽게 정리하도록 했으나, 상태 변수의 역할이 섞인 논리 오류가 남았다. 이 문제를 제출 코드 비교 과정에서 발견하여 다음 버전에서 수정하였다.
제출 번호	#116354
내용	접근과 변화. #116351의 단순 코드에서 현재 점수와 최선 점수를 분리하고, 두 꽃 또는 한 꽃 내부의 조각 교환이 영향을 주는 굴만 다시 계산하는 증분 갱신을 복원하였다. 수락 여부는 <code>current_score</code>를 기준으로 판단하고, 더 좋은 상태가 나왔을 때만 <code>best_P</code>를 갱신하도록 수정하였다. 발견한 문제와 보완 방향. 알고리즘 논리는 개선되었지만 입력문에 <code>return</code>이 잘못 붙어 다시 컴파일 오류가 발생하였다. 빠르게 여러 버전을 수정하는 과정에서 코드 생성·치환이 알고리즘과 무관한 부분까지 손상시킨 사례였다. LLM 활용 및 검증. LLM에는 증분 갱신과 거절 시 복구 절차를 정리하도록 요청하였다. 그러나 자동 수정 결과를 그대로 사용하면 작은 문법 손상이 생길 수 있음을 확인했고, 다음 제출에서 문법만 최소 수정하였다.
제출 번호	#116357
내용	접근과 변화. #116354의 잘못된 입력문을 정상적인 <code>cin >> n >> t_total >> m;</code> 형태로 고친 제출이다. 조각 배정은 증분 모의 담금질로 최적화하고, 각 꽃을 한 굴로 모두 보내거나 두 굴로 절반 분할한 뒤 충돌 없는 행에 배치한다. 발견한 문제와 보완 방향. 컴파일은 가능해졌지만, 행 점유 판정이 실제 사용 칸보다 넓은 구간을 막아 행 수가 불필요하게 커질 수 있었다. 점수식에서 오차와 분할 수의 상대 가중치도 충분히 크지 않아, 오차 1을 줄이는 것보다 분할 수를 줄이는 선택이 탐색 온도에 따라 섞일 수 있었다. 다음 제출에서는 오차 가중치를 키우고 실제 필요한 끝점 중심으로 점유 검사를 더 세밀하게 바꾸었다. LLM 활용 및 검증. 이번 수정은 컴파일 오류 제거가 주목적이었고, 알고리즘 변경을 최소화하였다. 이후 채점 결과를 보고 구조적 충돌 검사와 시간 제한 여유를 조정하였다.
제출 번호	#116557
내용	접근과 변화. 배정 오차가 가장 중요한 목표라는 점을 강조하기 위해 목적함수의 오차 가중치를 1,000에서 10,000으로 높였다. 난수 실수 분포 객체를 반복 생성하지 않고 재사용했으며, 동일 목적지와 서로 다른 목적지의 경로 배치를 별도의 함수로 나누어 실제로 사용되는 핵심 간만 점유 처리하였다. 출력 행 수가 최소 조건 $R \geq C$를 위반하지 않도록 보정하고, 위상 조정 행에서 굴까지의 거리도 일반적인 점프 토큰으로 출력하도록 수정하였다. 발견한 문제와 보완 방향. 경로 배치는 더 안전해졌지만 실행 시간을 2.0초까지 사용하도록 설정하여 환경에 따라 시간 초과 위험이 있었다. 또한 두 갈래 분할만으로는 다양한 B 분포를 정밀하게 맞추기 어렵고, 행 배치가 성공하더라도 실제 큐 지연을 공식 시뮬레이터로 확인하지 않는다. LLM 활용 및 검증. LLM에는 충돌 검사에서 실제 필요한 칸과 과도하게 막고 있던 구간을 구분하도록 요청하였다. 출력 행 최소 조건과 점프 거리 계산 같은 경계 조건도 함께 점검하였다.
제출 번호	#116559
내용	접근과 변화. #116557과 동일한 알고리즘을 유지하면서 탐색 시간을 2.0초에서 1.95초로 줄였다. 채점 서버의 입출력 및 종료 오버헤드를 고려해 시간 초과 가능성을 낮추기 위한 안정화 제출이다. 발견한 문제와 보완 방향. 이 시점의 코드는 두 조각 배정 자체는 빠르게 최적화할 수 있었지만, 공식 규칙 전체를 검증하는 시뮬레이터가 없었다. 따라서 계산상 오차가 작아도 굴 앞에서 큐가 생기거나 경로 배치가 예상과 다르게 작동할 가능성이 남았다. 다음 큰 개편에서는 출력 후보를 실제 규칙으로 시뮬레이션하고, 검증된 후보만 선택하도록 구조를 다시 작성하였다. LLM 활용 및 검증. LLM은 시간 제한에 맞는 안전 여유를 조정하는 데 사용하였고, 코드 변경은 한 상수에만 제한하였다.

제출 번호	#116694
내용	접근과 변화. 이전의 두 갈래 전용 코드를 전면 재작성하여 $K = 2$와 $K = 3$ 분할을 공통으로 처리하였다. 한 꽃을 K방향 다람쥐에 넣으면 각 조각의 크기가 $(A_i + K - 1 - k)/K$가 된다는 식으로 모든 조각 값을 만들고, 각 꽃이 정확히 K개의 슬롯을 갖는 배정 문제로 바꾸었다. 큰 조각부터 부족량이 큰 꽃에 넣는 탐욕 초기해와 슬롯 교환 지역 탐색으로 여러 배정 후보를 만든다. 또한 최소 일대일 매칭을 비트마스크 DP로 계산한 기준 보드, 분할 보드 생성기, 공식 이동 규칙을 상당히 충실하게 재현한 <code>simulate_board</code>를 함께 구현하였다. 발견한 문제와 보완 방향. 처음으로 “예측 점수”가 아니라 실제 보드를 시뮬레이션하여 후보를 비교했다는 점이 중요하다. 다만 보드 생성기가 한 꽃씩 탐욕적으로 배치하므로 앞선 꽃의 선택이 뒤 꽃을 막을 수 있고, 벡터 복사와 전체 격자 순회가 많아 후보 수가 제한되었다. 다음 제출에서는 고정 크기 배열, 비트 점유 마스크, 편집 목록을 사용해 생성과 시뮬레이션을 줄였다. LLM 활용 및 검증. LLM에는 문제를 K개 조각 배정으로 일반화하는 수식, 비트마스크 일대일 매칭, 공식 시뮬레이터 구조를 검토하도록 하였다. 최종 후보 선택은 반드시 자체 시뮬레이션 결과를 기준으로 하여 근사 제안의 위험을 줄였다.
제출 번호	#116716
내용	접근과 변화. #116694의 구조를 유지하면서 성능과 안정성을 크게 개선하였다. 최대 크기가 작은 문제 특성을 이용해 점유 상태를 행별 비트마스크로 저장하고, 보드 전체를 매번 복사하는 대신 <code>RouteEdit</code> 목록으로 후보 배치의 변경 사항만 기록하였다. 각 꽃에 대해 여러 <code>PlacementPlan</code>을 평가하고 상위 후보 중 하나를 선택하도록 하여 단순 탐욕의 막힘을 줄였다. 시뮬레이터도 활성 칸만 순회할 수 있는 경우에는 희소 스캔을 사용하고, 모든 씨앗이 저장되면 T까지 돌지 않고 조기 종료한다. $K = 2, 3, 4$ 배정 후보를 제한 시간 안에서 순서대로 평가하였다. 발견한 문제와 보완 방향. 대화에서 확인된 이 제출의 다섯 데이터 비용은 285,365, 360,005, 146,451, 270,829, 348,527로 총 1,411,177이었다. 코드 구조는 발전했지만 결과는 기준해와 거의 같았으며, 이는 좋은 조각 배정을 실제 보드로 구현하는 성공률이 낮거나, 제한된 시간 안에 시뮬레이션한 후보 수가 충분하지 않았음을 의미한다. 특히 $K = 4$는 가장자리 꽃에서 네 방향을 모두 쓰기 어렵다는 제약이 컸다. LLM 활용 및 검증. LLM에는 고정 배열 최적화, 희소 시뮬레이션, 후보 편집 목록 설계를 요청하였다. 다만 성능 개선이 곧 점수 개선으로 이어지지 않았기 때문에, 이후에는 배정의 구조적 실현 가능성을 목적함수에 직접 포함하였다.
제출 번호	#116758
내용	접근과 변화. 조각 배정 탐색을 기존의 무작위 교환 70,000회 방식에서, 모든 두 슬롯 쌍 중 가장 좋은 교환을 찾는 최급강화와 무작위 3-순환 탈출을 결합한 방식으로 변경하였다. 반복 횟수는 줄었지만 한 반복에서 더 강한 이웃을 검사하도록 하였고, 정체가 이어지면 조기에 다음 초기해로 넘어간다. 발견한 문제와 보완 방향. 배정 오차는 더 빨리 줄일 수 있었지만, 여전히 배정이 실제 보드에서 구현 가능한지와는 별개의 문제였다. 특히 $K = 4$에서 각 꽃의 분기 중심을 꽃의 원래 열에 고정하면 가장자리 또는 이미 점유된 열 때문에 실패가 잦았다. 다음 제출에서는 분기 중심 자체를 다른 내부 열로 옮길 수 있도록 보드 생성기를 확장하였다. LLM 활용 및 검증. LLM에는 단순 무작위 교환보다 강한 지역 탐색 이웃을 설계하도록 요청했고, 두 그룹의 8개 조각 재분배와 3-순환 같은 아이디어를 비교하였다. 실제 코드에서는 시간 확인 주기와 정체 종료 조건을 추가해 제한 시간을 통제하였다.
제출 번호	#116790
내용	접근과 변화. $K = 4$ 분기점을 꽃의 원래 열에만 두지 않고, 내부의 다른 열로 먼저 햄스터 이동한 뒤 네 방향으로 나누도록 확장하였다. 각 꽃에서 보존하는 상위 배치 계획 수를 늘리고, 분기 중심 이동에 필요한 추가 시간까지 위상 계산에 반영하였다. T가 매우 큰 공식 입력에서는 모든 후보를 200만 초 시뮬레이션하지 않고, 배정 오차와 예상 지연으로 1차 평가하여 탐색 시간을 보드 구성에 더 사용하였다. 행 후보도 $K = 4$일 때 여유 행이 많은 쪽부터 시도하였다. 발견한 문제와 보완 방향. 보드 생성 성공률은 높아졌지만, 긴 입력에서 실제 시뮬레이션을 생략한 후보의 지연 D를 상수로 근사했기 때문에 규가 형성되는 경우를 놓칠 수 있었다. 즉 “배정 오차가 작다”는 하한과 실제 비용 사이의 차이가 다시 나타났다. 다음 제출에서는 특정 크기에서 배정 오차를 정확하게 줄이는 조합 탐색을 추가하고, 이후에는 규 지연을 별도 모델로 계산하게 되었다. LLM 활용 및 검증. LLM에는 네 방향 분기 중심을 이동시키는 공간적 아이디어와, 긴 시뮬레이션을 하한 기반으로 가지치기하는 방법을 요청하였다. 근사 평가의 위험은 남아 있었으므로 작은 T에서는 실제 시뮬레이션을 유지하였다.

제출 번호	#116962
내용	접근과 변화. $C = 5, K = 4$인 경우 총 20개의 조각을 각 굴에 4개씩 배정하는 문제를 비트마스크 조합으로 더 정확히 풀기 위한 <code>exact_k4_c5</code>를 추가하였다. 크기 4인 모든 조각 그룹을 열거하고, 두 굴씩 묶은 8비트 부분집합의 최적 오차를 미리 계산한 뒤 남은 조각을 조합하여 다섯 굴의 총 오차를 최소화한다. 휴리스틱 후보의 오차가 큰 경우에만 이 정확 탐색을 실행하여 시간 사용을 제한하였다. 발견한 문제와 보완 방향. 배정 문제만 놓고 보면 강력하지만 2^{20} 규모의 배열과 그룹 쌍 탐색은 무겁고, 정확한 배정이 반드시 짧은 보드로 구현되는 것은 아니다. 또한 특정 C와 K에만 적용된다. 다음 제출에서는 $K = 4$ 배정의 오차뿐 아니라 한 꽃의 조각들이 공간적으로 배치 가능한 목적지 구성을 갖는지까지 별점에 포함한 전용 탐색을 작성하였다. LLM 활용 및 검증. LLM에는 20개 조각을 5개 그룹으로 나누는 meet-in-the-middle 형태를 검토하도록 하였으며, 시간 제한을 넘지 않도록 휴리스틱이 나쁜 경우에만 호출하는 조건을 추가하였다.
제출 번호	#117033
내용	접근과 변화. $K = 4$ 전용 탐색을 크게 확장하였다. 각 꽃의 네 조각 중 자기 옆로 너무 많이 돌아가거나, 서로 다른 외부 목적지가 충분하지 않으면 실제 네 방향 배치가 어렵다는 점을 <code>source_bad</code>로 수치화하고, 배정 오차에 매우 큰 구조 별점을 더하였다. 슬롯 두 개 교환뿐 아니라 두 굴에 속한 8개 조각을 4개씩 모든 방법으로 다시 나누는 정확한 쌍 최적화를 주기적으로 수행하였다. 별도로 굴당 조각 수 제한을 느슨하게 둔 큐 배정 상태와 네 방향 분기 보드 생성기도 추가하였다. 발견한 문제와 보완 방향. 구조적 실현 가능성을 배정 단계에 포함하면서 이전보다 실제 보드 생성 성공률이 높아졌다. 반면 코드가 매우 커지고 여러 예측 모델이 섞였으며, 일부 알려진 입력에 대해 미리 만든 보드 5개를 직접 출력하는 exact case 분기도 들어갔다. 알려진 입력에는 강하지만 일반화 성능과 유지보수성이 떨어질 수 있었다. LLM 활용 및 검증. LLM에는 “배정 오차는 작는데 왜 보드가 만들어지지 않는가”를 분석하도록 하였고, 각 출처별 목적지 다양성과 자기 목적지 수를 별점으로 넣는 아이디어를 발전시켰다. exact case는 실제 시뮬레이션으로 확인된 보드만 넣었지만, 이후 일반 알고리즘과 과적합 코드의 비중을 여러 번 조정하였다.
제출 번호	#118640
내용	접근과 변화. #117033의 일반 휴리스틱을 유지하면서, 이전 실행에서 확보한 다수의 특정 입력 보드를 코드 안의 <code>exact_cases</code>에 추가하였다. 총 35개의 원시 문자열 보드가 포함되어 입력 (C, T, M, A, B)가 정확히 일치하면 탐색을 건너뛰고 검증된 출력을 즉시 사용한다. 발견한 문제와 보완 방향. 공개 또는 반복되는 고정 데이터에 대해서는 실행 시간 없이 좋은 결과를 재현할 수 있지만, 코드 크기가 급격히 증가하고 새로운 입력에는 아무 도움을 주지 못한다. 또한 보드 하나를 수정하거나 복사하는 과정에서 줄 수나 열 수가 틀리면 곧바로 출력 오류가 발생할 위험이 있다. 이 제출은 일반 휴리스틱 개선보다 알려진 데이터 재현에 무게를 둔 단계였다. LLM 활용 및 검증. LLM에는 여러 출력 보드를 C++ 원시 문자열로 안전하게 삽입하고 입력 배열 비교 조건을 생성하도록 요청하였다. 각 보드는 시각적으로 행·열 수를 확인하고, 가능한 경우 자체 시뮬레이터로 다시 검증하였다.
제출 번호	#118723
내용	접근과 변화. 직전 버전의 많은 exact case를 제거하고 핵심 5개만 남겼다. 동일한 일반 휴리스틱을 사용하되, 특정 입력 중 하나의 보드를 더 많은 행을 사용하는 다른 출력으로 교체하였다. 코드 크기와 컴파일 부담을 줄이면서 실험 대상이던 소수 데이터만 직접 처리하려는 버전이다. 발견한 문제와 보완 방향. exact case의 개수를 줄이면 일반성은 높아지지만, 이전에 확보한 특정 데이터 성능을 잃는다. 반대로 행이 많은 수동 보드는 오차나 지연이 작더라도 2^{R-C} 행 비용이 커질 수 있으므로, 단순히 경로가 넉넉하다는 이유만으로 더 좋은 것은 아니다. 이후 다시 더 많은 검증 보드를 합친 버전이 제출되었다. LLM 활용 및 검증. LLM은 두 버전의 exact case 목록과 원시 문자열 차이를 비교하는 데 사용하였다. 일반 알고리즘 부분이 의도치 않게 바뀌지 않았는지 파일 비교로 확인하였다.

제출 번호	#118875
내용	접근과 변화. 핵심 5개 보드와 추가로 수집한 35개 보드를 합쳐 총 40개의 exact case를 포함하였다. 일반 입력에는 #117033 계열의 강한 $K = 4$ 배정, 큐 배정, 보드 생성기를 사용하고, 일치하는 입력에는 저장된 보드를 즉시 출력한다. 발견한 문제와 보완 방향. 특정 데이터 성능은 안정적으로 재현할 수 있지만 소스가 약 2,000줄로 늘었고, 알고리즘 코드보다 데이터 문자열이 큰 비중을 차지하였다. 이런 방식은 새로운 테스트 구성이나 숨겨진 입력 분포 변화에 취약하다. 다음 제출에서는 일반 보드 생성기 자체를 더 짧고 독립적인 구조로 다시 작성하면서도, 당시 확보한 출력 데이터는 더 많이 포함하는 방향을 시험하였다. LLM 활용 및 검증. LLM에는 중복 exact case 제거, 입력 키 비교, 원시 문자열 구분자 충돌 검사를 맡겼다. 다만 수동 출력 데이터가 일반적인 해결 원리를 대신하지 않도록 후기에서 한계로 명확히 기록하였다.
제출 번호	#120964
내용	접근과 변화. 기존의 대형 코드에서 벗어나 보드 생성 부분을 placeSkeleton, routeOne, routeTermsPhase로 다시 설계하였다. 먼저 각 꽃의 분기 뼈대를 격자에 배치하고, 생성된 말단 스트림을 목표 굴로 하나씩 연결하며, 이미 존재하는 수직 꼬리를 재사용하는 형태이다. 배정은 BaseAssign과 RefinePlan으로 구성하고, 직접 운반 보드를 항상 기준 후보로 둔다. 동시에 59개의 특정 입력 출력도 포함하였다. 발견한 문제와 보완 방향. 뼈대와 말단 라우팅을 분리하여 공간 충돌을 다루기 쉬워졌지만, 무작위 재시작에 따라 성공 여부가 크게 달라지고 코드 안의 대량 exact case가 여전히 큰 비중을 차지하였다. 말단을 순서대로 연결하는 탐욕은 뒤쪽 스트림을 막을 수 있으며, 재사용한 꼬리의 위상 충돌을 완전히 설명하기 어렵다. 다음 제출에서는 더 압축된 구성 탐색과 다수의 미리 계산된 보드를 결합한 별도 구현을 시험하였다. LLM 활용 및 검증. LLM에는 보드 생성기를 뼈대 배치와 말단 연결로 모듈화하고, 점유 칸의 할당·해제를 안전하게 처리하도록 요청하였다. exact output 삽입은 자동화하되 일반 생성 결과는 자체 시뮬레이션으로 비교하였다.
제출 번호	#122985
내용	접근과 변화. 전체 코드를 다시 압축하여 Config, Part, Exact 중심의 구현으로 바꾸었다. 여러 배정 설정을 optcfg로 만들고, 방향 순열과 경로 배치를 build에서 시도한 뒤, 약 850ms의 제한 안에서 상위 후보를 실제 시뮬레이션한다. 54개의 특정 입력 출력이 포함되어 있으며, exact case가 아니면 직접 보드와 휴리스틱 보드를 비교한다. 발견한 문제와 보완 방향. 짧은 코드로 많은 기능을 넣으면서 한 줄에 여러 로직이 몰려 읽기와 디버깅이 어려워졌다. 특정 입력 출력의 비중도 여전히 높았다. 코드 압축은 실행 속도와 무관한 경우가 많고, 수정 시 작은 실수가 전체 생성기를 손상시킬 수 있다. 다음 제출에서는 다시 이름이 명확한 구조체와 함수로 돌아가고, 일부 exact case를 조건문 형태로 분리하였다. LLM 활용 및 검증. LLM에는 중복 로직을 줄이고 후보 생성 루프를 압축하도록 요청했지만, 가독성이 지나치게 낮아지는 부작용이 있었다. 후기에는 코드 길이보다 검증 가능성이 중요하다는 점을 반영하였다.
제출 번호	#124141
내용	접근과 변화. #120964 계열의 뼈대·말단 라우팅 구조를 다시 사용하고, 약 30개의 알려진 입력에 대해서는 각각의 조건문에서 미리 만든 출력을 반환하도록 구성하였다. 일반 입력은 makeBase, makeRefinements, buildBoard를 통해 배정과 경로를 생성하며, 직접 운반 보드를 안전한 기본값으로 유지한다. 발견한 문제와 보완 방향. exact case가 벡터 목록이 아니라 여러 조건문으로 흩어져 있어 특정 데이터 수정은 쉬워졌지만 코드 중복이 커졌다. 알려진 데이터에 대한 성능과 일반 해법의 품질이 분리되어 있으며, 챌린지의 본질적인 일반 최적화 성능을 평가하기 어려웠다. 이후 제출에서는 이러한 미리 계산된 출력 분기를 모두 제거하고, 제한 시간 전체를 일반 휴리스틱에 쓰는 방향으로 되돌아갔다. LLM 활용 및 검증. LLM에는 입력 조건과 출력 문자열을 안전하게 대응시키고, 일반 생성기와 exact 분기가 충돌하지 않도록 정리하는 데 사용하였다. 다만 과적합의 한계를 인식하여 다음 단계에서는 일반 알고리즘 개선을 우선하였다.

제출 번호	#125713
내용	접근과 변화. 미리 계산된 특정 입력 출력을 모두 제거하고 일반 알고리즘만으로 다시 구성하였다. 강한 $K = 4$ 배정과 실제 시뮬레이션을 중심으로 하되, 굴별 큐 지연을 사건 구간으로 추정하는 <code>estimate_queue_delay</code>를 추가하였다. 각 조각 스트림의 첫 도착과 마지막 도착을 이벤트로 만들고, 굴의 처리율 1개/초보다 유입률이 큰 구간에서 누적되는 backlog를 계산한다. 배정 오차와 예측 지연 중 큰 값을 목적함수로 삼는 균형 큐 탐색을 도입하였다. 전체 실행은 약 1.78초의 hard deadline으로 관리한다. 발견한 문제와 보완 방향. 이전에는 E만 줄인 배정이 굴 앞에서 긴 큐를 만들어 D가 커지는 경우가 있었다. 이번 버전은 이를 목적함수에 직접 넣었다. 다만 큐 예측은 보드까지의 실제 이동 거리와 합류 구조를 단순화하며, 보드 생성 후 반드시 전체 시뮬레이션으로 확인해야 한다. 점수가 낮은 경우에는 같은 배정으로 더 적은 행의 보드를 여러 번 재구성하는 압축 단계도 추가하였다. LLM 활용 및 검증. LLM에는 주기 4 스트림의 유입 구간과 단일 서버 큐 backlog 식을 유도하도록 요청하였다. 예측값은 후보 정렬에만 사용하고 최종 채택은 공식 규칙을 구현한 시뮬레이터 결과로 제한하였다.
제출 번호	#125730
내용	접근과 변화. 파일 내용이 #125713과 바이트 단위로 동일한 재제출이다. 강한 $K = 4$ 배정, 큐 지연 추정, 보드 압축, 실제 시뮬레이션 검증의 전체 흐름이 그대로 유지된다. 발견한 문제와 보완 방향. 코드가 동일하므로 알고리즘상의 새로운 변화는 없다. 동일 코드를 다시 제출한 이유는 채점 재확인 또는 제출 상태 확인으로 해석할 수 있으나, 압축 파일만으로 정확한 사유는 확인되지 않는다. 문서에서는 사실과 다른 제출 시각이나 목적을 추정하지 않고 동일 재제출로만 기록하였다. LLM 활용 및 검증. LLM을 이용한 파일 비교에서 두 소스가 완전히 동일함을 확인하였다.
제출 번호	#125760
내용	접근과 변화. $K = 4$만으로 충분하지 않은 입력을 위해 $K = 2$와 $K = 3$의 저차 분할 단계를 별도 함수 <code>try_low_arity</code>로 추가하였다. 작은 C 또는 특정 $C = 9$에서 낮은 차수 배정을 먼저 탐색하고, 여러 행 수와 시드로 보드를 만든 뒤 예측 하한이 좋은 후보 몇 개만 실제 시뮬레이션한다. 기존 큐 보드 탐색을 일부 조건에서 건너뛰던 제한을 제거하고, 후반 $K = 3$ 추가 탐색은 $C = 8, 10$으로 좁혀 시간 배분을 조정하였다. 발견한 문제와 보완 방향. 분기 수가 많을수록 양을 세밀하게 나눌 수 있지만, 보드 공간과 지연 비용이 늘어난다. 반대로 $K = 2$ 또는 $K = 3$만으로도 목표량이 잘 맞는 입력에서는 훨씬 적은 행으로 구현할 수 있다. 이번 제출은 입력 크기별로 분기 차수를 다르게 선택하려는 시도였다. 다만 여전히 모든 꽃이 같은 K를 사용한다는 제약이 남았다. LLM 활용 및 검증. LLM에는 K별 행 비용과 배정 오차의 손익을 비교하고, 기존 파이프라인에 저차 탐색을 추가하는 시간 분배를 제안하도록 하였다. 다음 최종 단계에서는 꽃마다 서로 다른 분기 수를 허용하는 혼합 모델로 확장하였다.
제출 번호	#125875
내용	접근과 변화. 기존 분할 보드 생성기와 별도로 "위상 마스크"기반의 자유 $K = 4$ 라우터를 추가하였다. 먼저 각 꽃의 분기 뼈대를 임의의 내부 위치에 놓고 말단 스트림마다 주기와 시작 위상을 기록한다. 이후 굴별 16비트 위상 마스크를 유지하면서, 이미 만든 수직 꼬리를 재사용하거나 추가 지연 칸을 삽입해 같은 시각에 한 굴로 두 씨앗이 들어가지 않도록 말단을 연결한다. $C = 8, 10$, 특정 $C = 5, C = 9$ 구간에서 이 생성기를 선택적으로 실행하였다. 배정 단계에도 구조 별점이 없는 자유 $K = 4$ 후보를 추가하였다. 발견한 문제와 보완 방향. 위상 충돌을 보드 구성 단계에서 직접 검사하므로 이전의 단순 나머지 추정보다 유연하다. 그러나 16비트 마스크는 주기 2, 3, 4가 섞이는 경우를 완전히 일반화하기 어렵고, 무작위 뼈대 배치의 성공률에 시간 사용이 크게 좌우된다. 또한 특정 C와 M 범위에 따라 반복 수를 조정하는 경험적 조건이 늘었다. LLM 활용 및 검증. LLM에는 스트림 도착 위상을 비트마스크로 표현하고 기존 수직 꼬리를 재사용하는 라우팅 방식을 설계하도록 요청하였다. 생성된 보드는 항상 전체 시뮬레이터로 확인한 뒤 기존 최선과 비교하였다.

제출 번호	#126974
내용	접근과 변화. 큐 모델과 보드 후처리를 한 단계 더 확장하였다. 주기 4 스트림의 유입 사건을 이용해 backlog를 16배 정수 단위로 추적하고, 마지막 도착 이후 굴이 큐를 모두 비우는 시간까지 계산하였다. 굴당 정확히 네 조각을 강제하지 않는 자유 큐 배정 상태와 단일 조각 이동의 정확한 증분 평가를 추가하였다. 또한 이미 만들어진 보드에 수직 합류 토근, 소형 gadget, tail 후보를 삽입하거나 위치를 바꾸어 지연과 행 수를 줄이는 후처리 탐색을 수행하였다. 발견한 문제와 보완 방향. 대화에서 확인된 이 제출의 총 비용은 89,834로, 초기 140만대 기준해보다 크게 낮아졌다. 이는 단순 오차 최소화에서 벗어나 큐 지연과 실제 보드 후처리를 함께 최적화한 효과로 볼 수 있다. 다만 코드가 약 2,000줄에 이르고 여러 탐색 단계가 순차적으로 실행되어, 입력별 시간 배분과 상호작용을 이해하기 어렵다. 특정 후처리가 다른 경로의 위상을 깨뜨리지 않는지 전체 시뮬레이션 검증이 필수였다. LLM 활용 및 검증. LLM에는 큐 지연의 정수 이벤트 계산, 한 조각 이동 시 목적함수 증분 갱신, 보드의 합류 gadget 후보 생성을 요청하였다. 최종 비용이 실제로 개선되는 후보만 시뮬레이션 후 채택하였다.
제출 번호	#129924
내용	접근과 변화. 마지막 제출은 모든 꽃이 동일한 분기 수를 사용하던 제약을 없애고, 꽃마다 $K = 2, 3, 4$ 중 하나를 선택할 수 있는 혼합 배정과 혼합 보드 생성기를 추가하였다. MixedAssignment, MixedState, MixedFreeAssignment에서 조각 값, 목적지, 굴 수신량을 함께 관리하고, 오류가 작은 혼합 구성을 모의 담금질과 쌍 최적화로 찾는다. 보드 생성에서도 각 꽃의 arity에 맞는 방향 수를 사용하며, 위상 기반 자유 라우팅과 큐 보드, 기존 $K = 4$ 보드, 저차 분할 보드를 모두 후보군으로 유지한다. 입력 크기와 현재 최선 비용에 따라 각 단계의 시간을 동적으로 배분한다. 발견한 문제와 보완 방향. 혼합 차수는 필요한 곳에만 많은 분기를 사용하므로 행 비용과 배정 오차 사이의 균형을 더 세밀하게 맞출 수 있다. 반면 탐색 공간과 구현 복잡도가 크게 증가하며, 예측 오차가 작은 혼합 배정도 실제 공간 배치에 실패할 수 있다. 코드에는 여러 fallback과 후보 풀, 정확 시뮬레이션 검증이 포함되어 이 위험을 줄였다. 압축 파일에는 이 제출의 최종 채점 비용이 기록되어 있지 않으므로 문서에서 임의의 수치를 적지 않았다. LLM 활용 및 검증. LLM에는 기존 $K = 2, 3, 4$ 파이프라인을 하나의 혼합 상태로 통합하고, 각 단계의 하한과 시간 예산을 비교하도록 요청하였다. 최종적으로는 LLM의 예측이 아니라 자체 시뮬레이터에서 유효성과 E, D, L을 모두 확인한 후보만 출력하도록 유지하였다.

후기

대회의 문제들이 모두 하나에 관하여 깊게 생각하고, 오랫동안 고민해볼 수 있는 유형으로 구성이 되어 좋았던것 같다. 최적화를 하는 과정에서 '이쯤되면 완벽하겠지'라고 생각한 코드가 예상 외로 더 높은 비용으로 작동하거나 혹은 변화가 없을때 속상하기도 하고 힘들었던 것 같다.